Anticipez les besoins en consommations de bâtiments

Projet 3 du parcours **Data Scientist Machine Learning**

Projet : Prédire les émissions de CO2 de bâtiments non destinés à l'habitation de la ville de Seattle

Objectif : Créer des variables pertinentes pour alimenter un modèle de ML supervisé

21/08/2026

1. L’entreprise

La ville de Seattle, a fait effectuer par ses agents des relevés minutieux de données énergétiques en 2016.

1.2 La mission

La ville doit atteindre son objectif de **ville neutre** en carbone d'ici 2050. Pour cela, en se basant sur ce qui a déjà été mesuré, il nous est demandé de prédire les **émissions de CO2** et la **consommation totale d’énergie** des bâtiments non résidentiels pour lesquels ces données n'ont pas encore été mesurées. »

La prédiction se basera sur les données structurelles des bâtiments (taille, usage, date de construction, situation géographique, et présence ou type d'énergie).

2. Exploration des données (EDA)

2.1 Utilisation de la fonction describe()

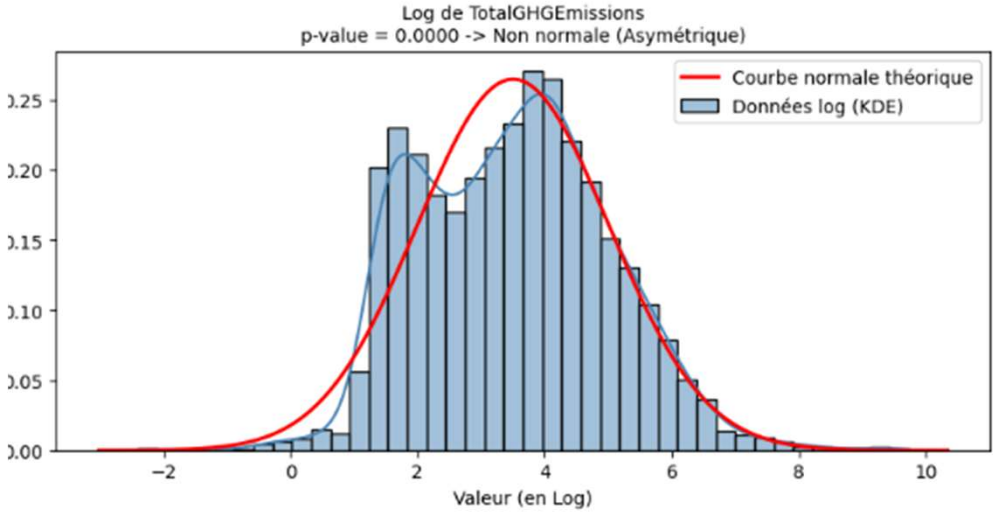
J'utilise la fonctions describe() de Pandas pour analyser la répartition des données du DataFrame (moyenne, médiane, quartiles), ainsi que leur dispersion et leur biais.

	OSEBuildingID	DataYear	ZipCode	CouncilDistrictCode	Latitude	Longitude	YearBuilt	NumberofBuildings	NumberofFloors	PropertyG
count	3376.000000	3.376000e+03	3360.000000	3376.000000	3376.000000	3376.000000	3376.000000	3368.000000	3376.000000	3.376(
mean	21208.991114	2.016000e+03	98116.949107	4.439277	47.624033	-122.334795	1968.573164	1.106888	4.709123	9.4833
std	12223.757015	0.000000e+00	18.615205	2.120625	0.047758	0.027203	33.088156	2.108402	5.494465	2.1883
min	1.000000	2.016000e+03	98006.000000	1.000000	47.499170	-122.414250	1900.000000	0.000000	0.000000	1.1285
25%	19990.750000	2.016000e+03	98105.000000	3.000000	47.599860	-122.350662	1948.000000	1.000000	2.000000	2.8487
50%	23112.000000	2.016000e+03	98115.000000	4.000000	47.618675	-122.332495	1975.000000	1.000000	4.000000	4.4175
75%	25994.250000	2.016000e+03	98122.000000	7.000000	47.657115	-122.319407	1997.000000	1.000000	5.000000	9.0992
max	50226.000000	2.016000e+03	98272.000000	7.000000	47.733870	-122.220966	2015.000000	111.000000	99.000000	9.3201
dispersion_%	57.634788	1.128013e-14	0.018972	47.769611	0.100282	-0.022237	1.680819	190.480072	116.677023	2.3075
skewness (biais)	-0.008279	0.000000e+00	1.999662	-0.070154	0.140045	-0.137526	-0.539445	43.394995	5.922340	2.4126

10 rows × 30 columns

2.2 Utilisation KDE et QQ-plot

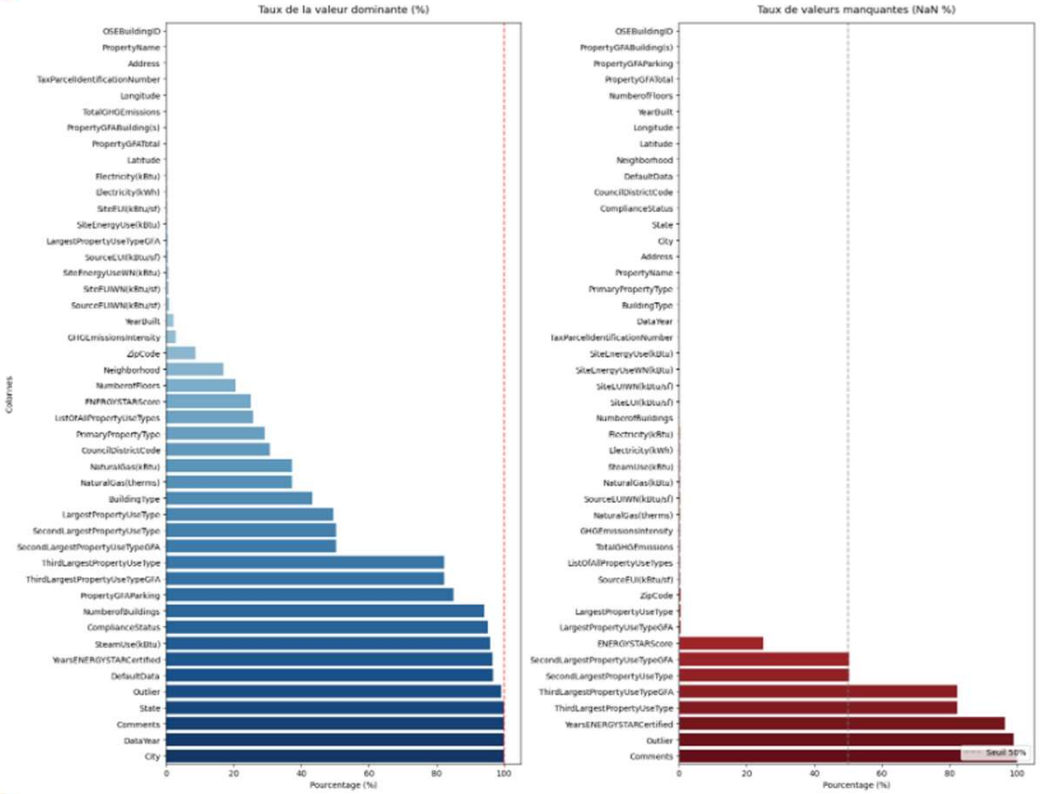
Je visualise la distribution de mes données avec une courbe KDE les graphiques QQ-plot et les tests statistiques pour vérifier la normalité.

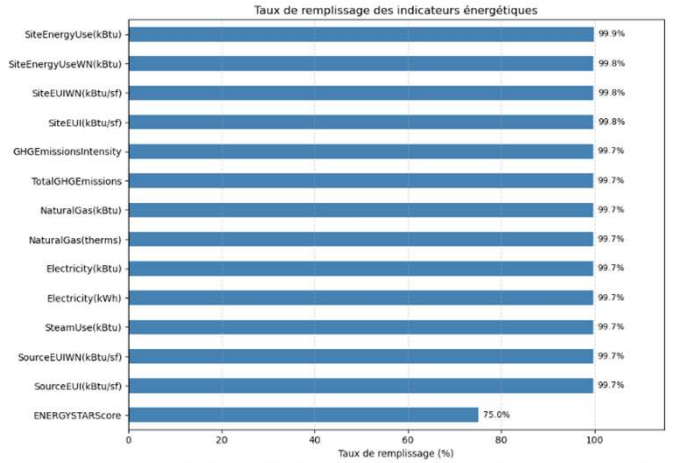


Dans le cas de non-normalité, j'ai des données asymétriques. Je redouble de vigilance lors du découpage train/test.

3.Visualisation du taux de valeurs dominantes et des valeurs manquantes

Je visualise les colonnes qui ont une valeur unique, donc inutiles pour l'apprentissage des modèles, ainsi que les bâtiments avec des données d'énergie manquantes (NaN).





PropertyName	ComplianceStatus	SiteEUI(kBtu/sf)	SiteEUIWN(kBtu/sf)	SiteEnergyUse(kBtu)	SiteEnergyUseWN(kBtu)	Electricity(kWh)	Electricity(kBtu)	N
Washington State Convention Center	Non-Compliant	NaN	0.000000	0.000000e+00	0.000000e+00	0.00	0.0	
Market Place I & II-2003 Western Avenue	Compliant	39.500000	NaN	5.177270e+06	NaN	1427645.25	4871126.0	
SEATTLE BUILDING	Non-Compliant	NaN	NaN	NaN	NaN	NaN	NaN	
SANDHURST APTS	Non-Compliant	NaN	NaN	NaN	NaN	NaN	NaN	
Forte Condominium	Non-Compliant	NaN	0.000000	0.000000e+00	0.000000e+00	0.00	0.0	
GRAHAM VIEW	Compliant	19.200001	20.200001	5.636832e+05	5.937986e+05	NaN	NaN	
MERCER CREST APT	Non-Compliant	NaN	NaN	NaN	NaN	NaN	NaN	
PONDERAY APTS	Compliant	27.100000	28.000000	7.702755e+05	7.975070e+05	NaN	NaN	
GRANVIEW APARTMENTS	Non-Compliant	NaN	NaN	NaN	NaN	NaN	NaN	
Villa Andora	Compliant	34.500000	37.000000	1.049639e+06	1.126782e+06	NaN	NaN	
KALBERG BUILDING	Non-Compliant	NaN	NaN	NaN	NaN	NaN	NaN	
STREAM FIFTEEN	Compliant	30.100000	31.500000	8.202201e+05	8.576127e+05	NaN	NaN	

4. Clean et création d'un nouveau DataFrame (df-clean)

En règle générale, les indicateurs d'énergie sont bien remplis, mais 12 bâtiments ont des valeurs manquantes.

Je crée la variable : **Batiment_without_data_energy**

Je supprime les colonnes aux valeurs constantes ou vides

DataYear, City, State, Comments

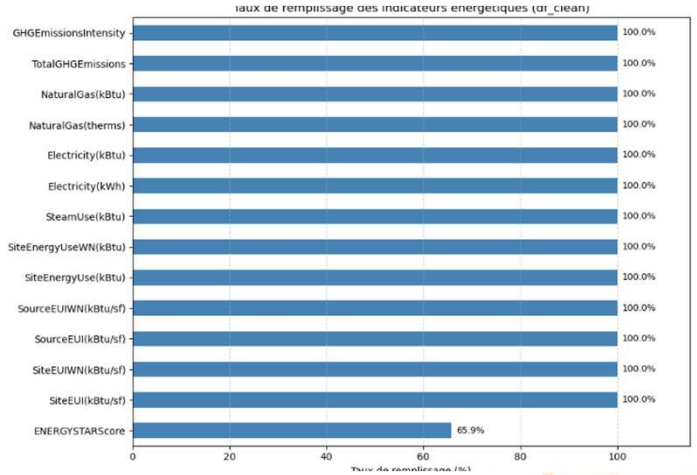
Je crée mes deux cibles

Cible 1 créée : **y1** → SiteEnergyUse(kBtu)
Cible 2 créée : **y2** → TotalGHGEmissions

Je crée le nouveau dataframe clean, **df-clean**

PropertyName	ComplianceStatus	SiteEUI(kBtu/sf)	SiteEUIWN(kBtu/sf)	SiteEnergyUse(kBtu)	SiteEnergyUseWN(kBtu)	Electricity(kWh)	Electricity(kBtu)	?
Washington State Convention Center	Non-Compliant	NaN	0.000000	0.000000e+00	0.000000e+00	0.00	0.0	
Market Place I & II-2003 Western Avenue	Compliant	39.500000	NaN	5.177270e+06	NaN	1427645.25	4871126.0	
SEATTLE BUILDING	Non-Compliant	NaN	NaN	NaN	NaN	NaN	NaN	
SANDHURST APTS	Non-Compliant	NaN	NaN	NaN	NaN	NaN	NaN	
Forte Condominium	Non-Compliant	NaN	0.000000	0.000000e+00	0.000000e+00	0.00	0.0	
GRAHAM VIEW	Compliant	19.200001	20.200001	5.636832e+05	5.937986e+05	NaN	NaN	
MERCER CREST APT	Non-Compliant	NaN	NaN	NaN	NaN	NaN	NaN	
PONDERAY APTS	Compliant	27.100000	28.000000	7.702755e+05	7.975070e+05	NaN	NaN	
GRANVIEW APARTMENTS	Non-Compliant	NaN	NaN	NaN	NaN	NaN	NaN	
Villa Andora	Compliant	34.500000	37.000000	1.049639e+06	1.126782e+06	NaN	NaN	
KALBERG BUILDING	Non-Compliant	NaN	NaN	NaN	NaN	NaN	NaN	
STREAM FIFTEEN	Compliant	30.100000	31.500000	8.202201e+05	8.576127e+05	NaN	NaN	

Après la suppression, mes données d'énergie n'ont plus de valeurs manquantes

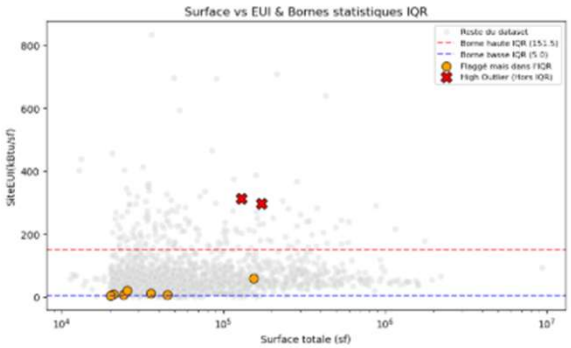


5. Modules

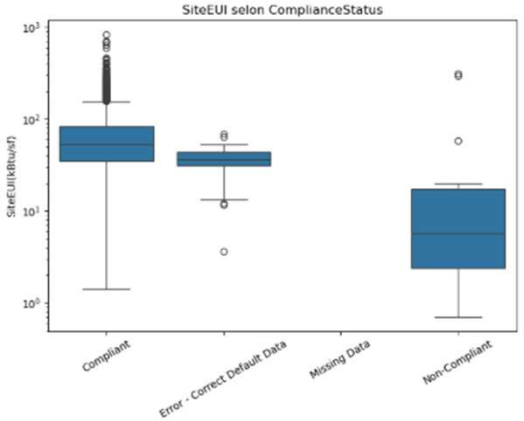
Je crée 4 modules portant le nom des colonnes que je vais étudier : *Outlier, Conformité, Energy Star et Certification, bâtiments mono et multi-usages*.

Ces modules me permettent d'étudier les colonnes du DataFrame par thèmes et de prendre des décisions à leur sujet.

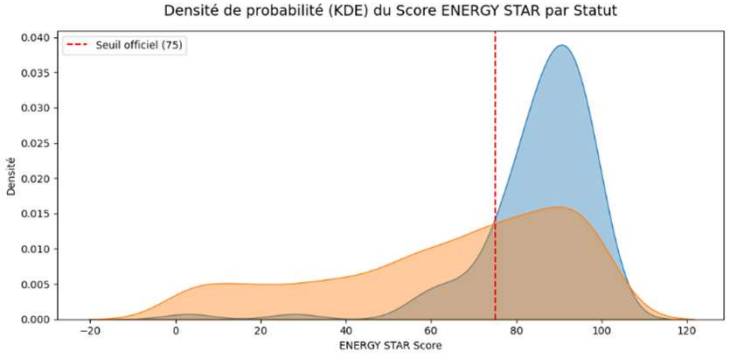
Un graphique module Outlier



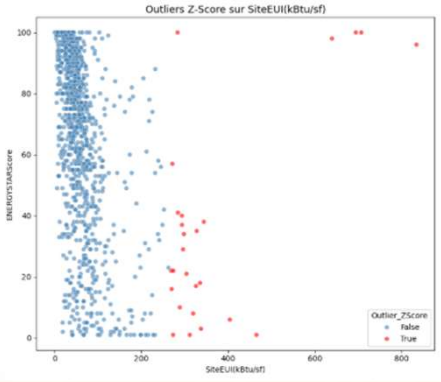
Un graphique du module conformité



Un graphique du module *Energy Star et Certification*



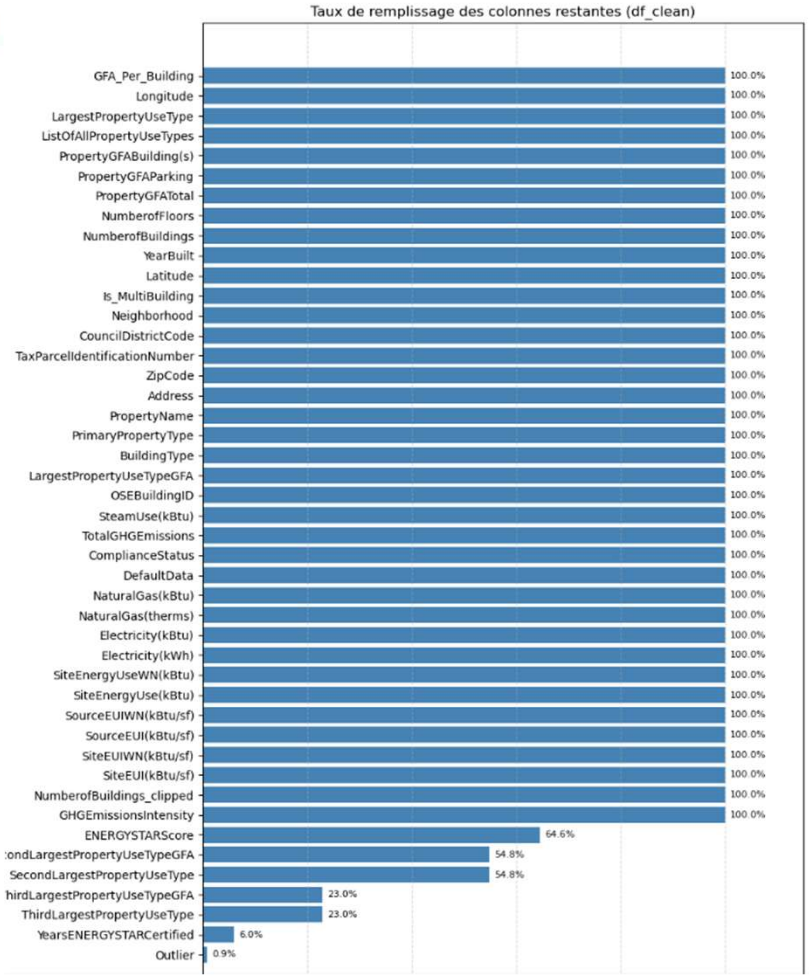
Un graphique du module bâtiments mono et multi-usages.



6. Décisions prisent concernant les colonnes étudiées

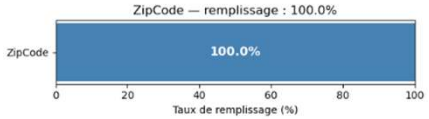
Élément analysé	Constat & Analyse	Décision & Justification
Outliers officiels (32 bât.)	23 Low / 9 High signalés par la ville. -1 pose problème sur Y_1, -0 sur Y_2	J'**exclus** l'ID 608, non conforme après analyse IQR et vérification manuelle des valeurs énergétiques.Je remarque que ces bâtiments identifiés comme *Outliers* ont un statut de conformité déclaré non conforme.
Conformité	Plusieurs status : Error / Flagged, Not Compliant, Compliant, Missing Data	J'**exclus** l'ensemble de ces bâtiments car ils sont déclarés comme non conformes par la ville. Batiments_non_compliance : 104
Energy Star & Certifs	Risque de *data leakage* (forte corrélation) et aussi incohérences (17 bâtiments certifiés avec un score de < 75). **Limitation climatique :** L'analyse des données de météo concentrée sur une seule année, ne permet pas de mesurer un impact climatique pluriannuel significatif.	**Exclus des *features* du modèle**. Le score ENERGY STAR et la certification ne sont pas conservés pour **éviter tout risque de *data leakage*** (fuite de données) et parce que la variable certification souffre de trop de valeurs manquantes et de décalages réglementaires.
Bâtiments mono et multi-usages	Répartition équilibrée du parc (604 mono-usages vs 526 multi-usages). Les bâtiments multi-usages affichent en moyenne une intensité énergétique (*SiteEUI*) plus élevée en raison de la complexité de leurs équipements. Morogénéité ou présence minoritaire (252 bâtiments avec parking vs 878 sans), affichant des impacts spécifiques sur la consommation d'énergie et l'intensité (*SiteEUI*)	Je crée des *features* binaires (**Has_SecondUse**, **Has_ThirdUse**) pour permettre au modèle de capturer l'impact de la mixité des usages sur la consommation. Je regroupe l'identification des structures complexes dans une variable (**NumberofBuildings**) et repère 55 *outliers* qui sont des mastodontes consommateurs d'énergie (Data Centers, hôpitaux, etc.).

7. Analyse des NaN restants



Les données manquantes vont être corrigé au cas par cas

-13 NaN concernent l'absence de **Zip code** qui vont être renseigné grâce a la bibliothèque geopy.geocoders qui a permit de renseigner les zip code manquant grâce a leur localisation.



-4 bâtiments n'avaient pas de **LargestPropertyUseType / GFA**.
Pour 3 d'entre eux il a été reconstruit grâce a le description PrimaryPropertyType, celui qui n'a pas pu être reconstruit a été supprimé.

8. Préparation des Features

Je prépare mes future comme suis :

1-**Renomage des variables** pour uniformiser les unités et les libellés (ajout du suffixe sf par exemple pour les surfaces)

2-**Géstion des valeurs manquantes** en remplaçant les cellules vides des usages secondaires, tertiaire et leur surface associer écrite « None » par 0.

3-**Création de nouvelles features**, contextuelle et temporelles (ex: BuildingAge = 2016 – YearBuilt.)

4-**Traitement anti-outliers** via le calcul d'un seuil au 99e percentile (max_bldg) pou rneutraliser les valeurs extrêmes et créer la colonne NumberofBuildings qui plafonne les valeurs trop élevées.

5-**Création de flags binaires pour les bâtiments atypiques** en fusionnant et en analysant les textes des colonnes d'usage (via Regex) je récupère les profils spécifiques à forte intensité énergétique (hôpitaux, campus universitaires, laboratoires, data centers et commerces alimentaires) → création d'une colonne par catégorie.

6-**Filtrage minimal du périmètre** pour éliminer les lignes aberrantes ou invalides (bâtiments avec nombre d'étage nul, consommation d'énergie a 0 ou émission GES négative).

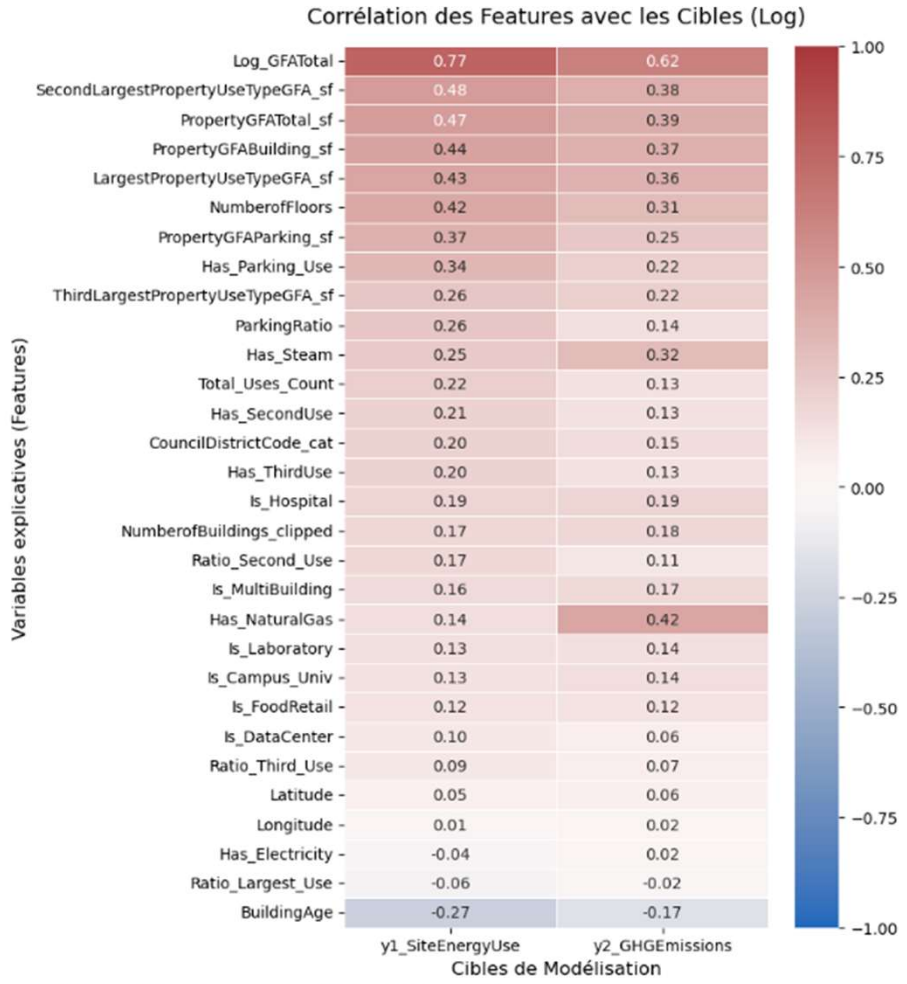
7-**Définition et transformation des cibles** y1 e ty2 en appliquant une transformation (log1) pour lisser la distribution.

8-**Création de X** avec deux listes de features concaténées, la liste des numéraires et celle des catégorielles.

9- **Mise en place d'un contrôle de cohérence (désalignement)** Création d'un message d'erreur en cas de désalignement des index entre les features (X) et les cibles (y).

9. Etude des corrélations vers cible y1 et y2

Réalisation d'une analyse de corrélation pour identifier la relation entre la variable explicatives et les cibles (y1 et y2)



10. Préparation du jeu train test et encodage

Le découpage et l'encodage via un pipeline garantissent l'absence de fuite de données (data leakage) et une évaluation réaliste des modèles.

Le découpage en amont (Train / Test) : On sépare le jeu de données brut en un ensemble d'entraînement (80%) et un ensemble de test (20%) avant toute modification.
Le jeu de test est mis de côté et reste strictement invisible pour s'assurer d'évaluer le modèle sur des données qu'il n'a jamais vues.
L'encodage dans le Pipeline : Au lieu d'encoder toutes les données d'un coup (ce qui ferait "fuir" des informations du test vers le train), on confie l'encodage et la mise à l'échelle à un `ColumnTransformer` intégré dans un `Pipeline`.
Comment ça fonctionne en pratique : Lors de la validation croisée ou de l'entraînement, l'encodeur apprend les catégories (fit) uniquement sur le sous-jeu d'entraînement de chaque pli, puis transforme le train et le test. Cela reproduit exactement les conditions réelles d'utilisation où de nouvelles données arrivent sans que le modèle ne les connaisse à l'avance.
La validation croisée (Cross-Validation 5-fold) : Réalisée en amont sur le jeu d'entraînement (X_train), elle découpe ce dernier en 5 plis pour entraîner et valider itérativement chaque modèle. Cela permet d'obtenir un score R2 moyen robuste (R2_cv) et de s'assurer que la performance du modèle ne dépend pas d'un découpage aléatoire unique.

11.Comparaison des modèles

En intégrant (PropertyGFATotal) variable la plus corrélé aux cibles y1et y2

Cible y1 — Consommation d'énergie

Variable d'origine : **SiteEnergyUse_kBtu** (kBtu) | Transformation active : **log1p**

Model	R2_cv	R2_test	MAE_log	RMSE_log	MAE_kBtu	RMSE_kBtu	Duration_s
SVR	0.497	0.525	0.603	0.817	4953847.599	26357816.138	1.239
XGBoost	0.535	0.523	0.598	0.819	4863189.532	26876822.429	1.044
GradientBoosting	0.531	0.521	0.594	0.820	4822617.236	26865360.758	3.586
ElasticNet	0.490	0.516	0.611	0.825	4850633.279	26665034.857	0.201
Ridge	0.494	0.512	0.611	0.828	4837398.919	26580229.940	0.136
Lasso	0.481	0.507	0.620	0.832	4910730.302	26757781.187	0.147
LinearRegression	0.503	0.479	0.638	0.856	5145592.734	26810192.375	0.138
RandomForest	0.484	0.472	0.635	0.861	5209569.600	27253655.626	3.037
Dummy_Mean	-0.011	-0.003	0.932	1.188	6476160.149	28013201.463	0.135

Meilleur modèle pour **Consommation d'énergie : SVR** (R^2 test = 0.525, erreur moyenne ≈ 4,953,848 kBtu)

Cible y2 — Émissions de GES

Variable d'origine : **TotalGHGEmissions_MTCO2e** (MTCO2e) | Transformation active : **log1p**

Model	R2_cv	R2_test	MAE_log	RMSE_log	MAE_MTCO2e	RMSE_MTCO2e	Duration_s
GradientBoosting	0.580	0.572	0.666	0.873	136.580	973.973	4.658
XGBoost	0.583	0.567	0.665	0.878	135.690	976.423	1.630
ElasticNet	0.526	0.551	0.679	0.894	134.107	970.216	0.231
Lasso	0.521	0.550	0.683	0.895	134.827	970.771	0.179
Ridge	0.520	0.543	0.681	0.902	134.603	970.340	0.171
SVR	0.533	0.529	0.691	0.916	137.809	972.191	1.014
LinearRegression	0.524	0.510	0.719	0.934	141.974	973.102	0.174
RandomForest	0.511	0.467	0.735	0.974	144.224	985.467	3.257
Dummy_Mean	-0.008	-0.007	1.042	1.339	162.815	995.479	0.193

Meilleur modèle pour **Émissions de GES : GradientBoosting** (R^2 test = 0.572, erreur moyenne ≈ 137 MTCO2e)

En intégrant la version de la variable passé en log (Log_GFATotal)

Cible y1 — Consommation d'énergie

Variable d'origine : **SiteEnergyUse_kBtu** (kBtu) | Transformation active : **log1p**

Model	R2_cv	R2_test	MAE_log	RMSE_log	MAE_kBtu	RMSE_kBtu	Duration_s
Ridge	0.728	0.749	0.447	0.593	3341893.804	20103111.490	0.168
LinearRegression	0.728	0.742	0.458	0.602	3464500.144	19805058.027	0.166
ElasticNet	0.724	0.738	0.459	0.607	3542756.239	21676483.015	0.172
Lasso	0.716	0.725	0.471	0.622	3709487.564	22813756.811	0.167
SVR	0.718	0.708	0.466	0.640	3808083.904	25182076.886	1.273
XGBoost	0.714	0.701	0.494	0.649	3906869.296	24114300.429	1.200
GradientBoosting	0.716	0.696	0.497	0.653	3911403.783	23892697.439	5.067
RandomForest	0.611	0.593	0.554	0.756	4808450.997	26824832.975	3.603
Dummy_Mean	-0.011	-0.003	0.932	1.188	6476160.149	28013201.463	0.141

Meilleur modèle pour **Consommation d'énergie : Ridge** (R^2 test = 0.749, erreur moyenne ≈ 3,341,894 kBtu)

Cible y2 — Émissions de GES

Variable d'origine : **TotalGHGEmissions_MTCO2e** (MTCO2e) | Transformation active : **log1p**

Model	R2_cv	R2_test	MAE_log	RMSE_log	MAE_MTCO2e	RMSE_MTCO2e	Duration_s
Ridge	0.680	0.692	0.574	0.741	120.258	916.399	0.183
ElasticNet	0.682	0.690	0.575	0.744	119.890	929.317	0.207
Lasso	0.679	0.683	0.580	0.751	120.019	933.982	0.189
LinearRegression	0.679	0.681	0.594	0.754	123.337	912.352	0.181
GradientBoosting	0.708	0.674	0.594	0.762	122.090	918.391	4.926
XGBoost	0.706	0.670	0.598	0.767	125.428	939.286	1.280
SVR	0.688	0.660	0.592	0.778	121.720	952.270	1.369
RandomForest	0.596	0.546	0.683	0.899	139.582	979.715	3.515
Dummy_Mean	-0.008	-0.007	1.042	1.339	162.815	995.479	0.184

Meilleur modèle pour **Émissions de GES : Ridge** (R^2 test = 0.692, erreur moyenne ≈ 120 MTCO2e)

Pour y1 : Le meilleur modèle atteint **R2_test = 0.525**, donc explique un peu plus de la moitié de la variance de la consommation d'énergie → correct mais pas exceptionnel pour ce genre de données bâtiment.
L'écart entre **MAE_log (0.603)** et **RMSE_log (0.817)** montre que le RMSE est nettement plus élevé que MAE, ce qui signale que les modèles ont **quelques grosses erreurs ponctuelles** (des bâtiments très mal prédits) plutôt qu'une erreur uniformément répartie → lié a des outliers massifs.

Les model prédictif sous forme d'arbre sont plus performants sur des features en grande partie faite de donnée binaires, ne permettant pas une analyse linaire.

12.Modification des features

Je retire plusieurs features binaires et j'obtient le résultat suivant

Cible y1 — Consommation d'énergie

Variable d'origine : **SiteEnergyUse_kBtu** (kBtu) | Transformation active : **log1p**

Model	R2_cv	R2_test	MAE_log	RMSE_log	MAE_kBtu	RMSE_kBtu	Duration_s
SVR	0.728	0.703	0.477	0.646	3782391.687	25007869.591	0.761
LinearRegression	0.724	0.729	0.472	0.618	3748103.861	23557337.652	0.106
XGBoost	0.715	0.677	0.509	0.673	4051644.091	24660807.376	1.171
GradientBoosting	0.713	0.675	0.511	0.675	4120055.179	24730630.283	1.919
Ridge	0.700	0.703	0.495	0.646	3954233.586	24306675.794	0.114
ElasticNet	0.692	0.691	0.505	0.659	3963325.996	24158322.909	0.094
Lasso	0.686	0.681	0.511	0.669	4009549.810	24176608.722	0.103
RandomForest	0.680	0.658	0.521	0.693	4608934.161	26160111.361	3.217
Dummy_Mean	-0.011	-0.003	0.932	1.188	6476160.149	28013201.463	0.074

Meilleur modèle pour **Consommation d'énergie : SVR** (R^2 test = 0.728, erreur moyenne ≈ 3,782,392 kBtu)

Cible y2 — Émissions de GES

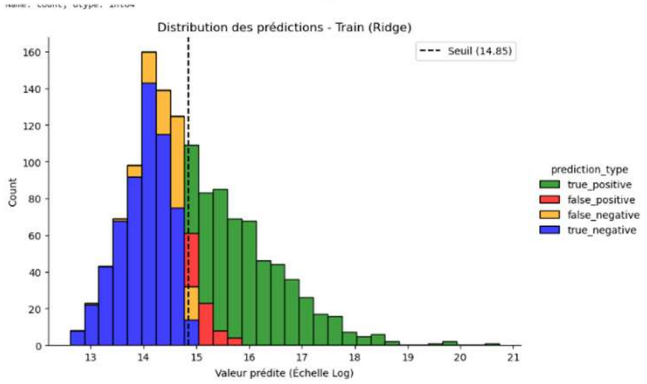
Variable d'origine : **TotalGHGEmissions_MTCO2e** (MTCO2e) | Transformation active : **log1p**

Model	R2_cv	R2_test	MAE_log	RMSE_log	MAE_MTCO2e	RMSE_MTCO2e	Duration_s
SVR	0.718	0.670	0.586	0.766	121.468	953.105	1.052
XGBoost	0.709	0.648	0.614	0.792	128.015	947.574	0.977
GradientBoosting	0.707	0.651	0.613	0.788	128.786	939.954	1.786
LinearRegression	0.702	0.680	0.588	0.755	123.768	931.012	0.125
Ridge	0.677	0.656	0.605	0.782	126.395	945.301	0.118
RandomForest	0.675	0.625	0.632	0.817	134.811	968.878	3.572
ElasticNet	0.670	0.645	0.615	0.795	125.474	944.763	0.150
Lasso	0.657	0.633	0.622	0.809	125.324	943.815	0.151
Dummy_Mean	-0.008	-0.007	1.042	1.339	162.815	995.479	0.120

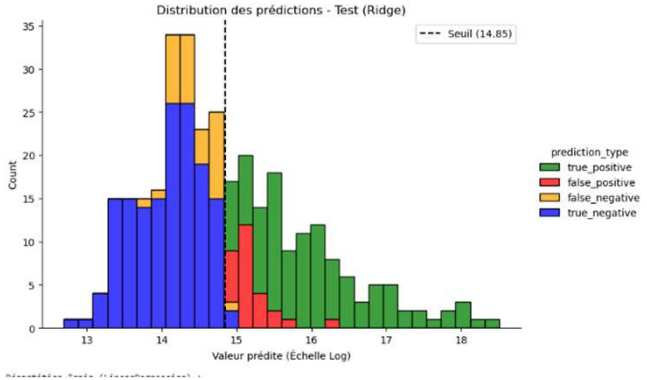
Meilleur modèle pour **Émissions de GES : SVR** (R^2 test = 0.718, erreur moyenne ≈ 121 MTCO2e)

L'on voit que les modèles linéaire telque LinearRegression ou SVR obtiennent de meilleurs résultats.

12. Analyse des erreurs : où et pourquoi le modèle se trompe

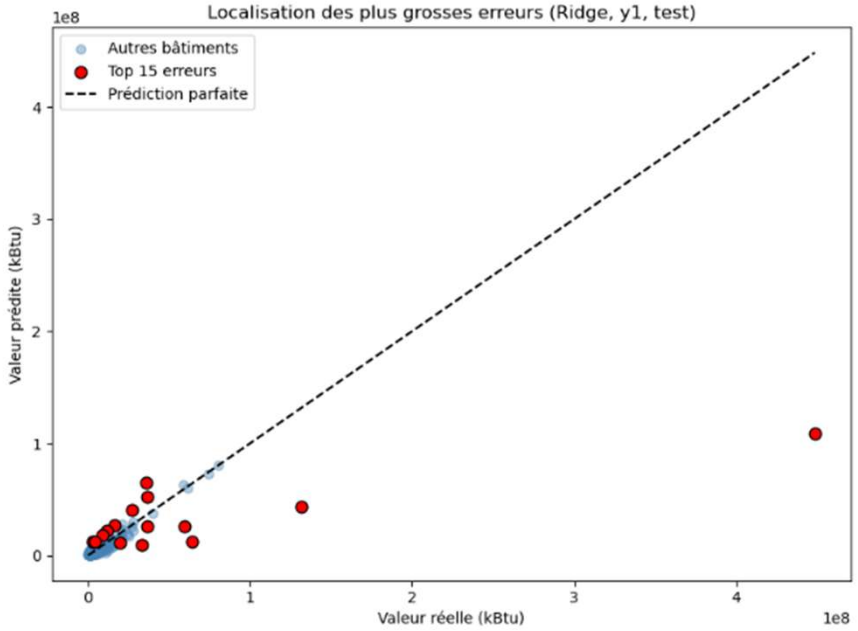



```
Répartition Test (Ridge) :
prediction_type
true_negative     153
true_positive     111
false_negative     33
false_positive     26
Name: count, dtype: int64
```



Sur-représentation de certains types de bâtiments dans les grosses erreurs

	% dans Top 15 erreurs	% dans le reste du test
Is_Hospital	20.0	3.2
Is_Campus_Univ	13.3	2.3
Is_Laboratory	6.7	0.3
Is_DataCenter	0.0	0.0
Is_FoodRetail	0.0	3.6



NB : Le log écrase les écarts entre grandes valeurs : en échelle log, 500 000 et 1 000 000 pieds carrés se ressemblent presque, alors que la différence reste énorme en consommation réelle. C'est ce qui peut faire sous-estimer le campus multi-bâtiments (448M kBtu réels vs ~109M prédits). Je vais alors garder la également la valeur brute à côté du log pour permettre au modèle de capter aussi cette amplitude réelle sur les cas extrêmes.

13.Analyse pour correction du biais

•**Avant la désactivation des variables biaisées (fortements corrélé)s :** Le modèle souffrait d'un biais massif de sous-estimation systématique et commettait des erreurs fortement prévisibles liées à la structure des bâtiments (fortes corrélations).
•**Après :** Le modèle alterne de façon équilibrée entre surestimations et sous-estimations. Ses erreurs ne dépendent plus de variables précises, ce qui prouve qu'il est devenu beaucoup plus stable.

	Residuals	Abs_Residuals
Abs_Residuals	1.000	1.000
Residuals	1.000	1.000
NumberofFloors	0.266	0.266
Has_SecondUse	0.255	0.255
Has_Parking_Use	0.229	0.229
Total_Uses_Count	0.198	0.198
Log_GFATotal	0.175	0.175
Ratio_Second_Use	0.136	0.136
YearBuilt	0.134	0.134
Longitude	0.125	0.125

	Residuals	Abs_Residuals
Abs_Residuals	0.035	1.000
Ratio_Third_Use	0.068	0.137
BuildingAge	0.018	0.106
Ratio_Largest_Use	0.088	0.064
Is_MultiBuilding	-0.085	0.039
Residuals	1.000	0.035
Has_NaturalGas	-0.061	0.014
Ratio_Second_Use	0.096	0.012
NumberofBuildings	-0.020	-0.003
Has_ThirdUse	0.077	-0.024

14.Recherche d'hyper paramètres

Hyperparamètre toutes features conservées

Résultats finaux pour y1 (Unité : kBtu) :

	Model	Initial_R2_Test	Best_R2_CV	Best_Params	MAE_kBtu	RMSE_kBtu	Biais_kBtu	Dispersion_kBtu
5	XGBoost	0.701	0.734	{'model__colsample_bytree': 0.8, 'model__learning_rate': 0.05, 'model__max_depth': 3, 'model__n_estimators': 200, 'model__subsample': 0.8}	3,967,938	24,369,080	2,368,715	24,253,685
2	ElasticNet	0.738	0.731	{'model__alpha': 0.001, 'model__l1_ratio': 0.9}	3,353,474	19,541,776	1,519,503	19,482,610
3	Lasso	0.725	0.731	{'model__alpha': 0.001}	3,348,506	19,487,077	1,518,088	19,427,856
0	Ridge	0.749	0.730	{'model__alpha': 1.0}	3,391,465	19,644,611	1,489,214	19,588,082
1	LinearRegression	0.742	0.728	{'model__fit_intercept': True}	3,464,500	19,805,058	1,451,975	19,751,762
6	GradientBoosting	0.696	0.727	{'model__learning_rate': 0.05, 'model__max_depth': 3, 'model__n_estimators': 200, 'model__subsample': 0.8}	3,895,340	23,214,017	2,214,493	23,108,150
4	SVR	0.708	0.722	{'model__C': 100, 'model__gamma': 0.001}	3,367,711	19,739,394	1,641,498	19,671,023
7	RandomForest	0.593	0.640	{'model__max_depth': 10, 'model__max_features': 'sqrt', 'model__min_samples_split': 2, 'model__n_estimators': 500}	4,680,368	26,617,048	3,532,855	26,381,550
8	Dummy_Mean	-0.003	-	-	6,476,160	28,013,201	4,626,670	27,628,489

Résultats finaux pour y2 (Unité : MTCO2e) :

	Model	Initial_R2_Test	Best_R2_CV	Best_Params	MAE_MTCO2e	RMSE_MTCO2e	Biais_MTCO2e	Dispersion_MTCO2e
2	XGBoost	0.685	0.720	{'model__colsample_bytree': 0.8, 'model__learning_rate': 0.05, 'model__max_depth': 3, 'model__n_estimators': 200, 'model__subsample': 0.8}	125	937	98	932
4	GradientBoosting	0.680	0.719	{'model__learning_rate': 0.05, 'model__max_depth': 3, 'model__n_estimators': 200, 'model__subsample': 0.8}	128	936	98	931
5	Lasso	0.670	0.703	{'model__alpha': 0.001}	123	892	82	889
3	ElasticNet	0.682	0.703	{'model__alpha': 0.001, 'model__l1_ratio': 0.9}	123	893	81	889
1	Ridge	0.694	0.702	{'model__alpha': 1.0}	124	892	80	889
6	SVR	0.646	0.701	{'model__C': 10, 'model__gamma': 0.01}	124	951	83	948
0	LinearRegression	0.697	0.699	{'model__fit_intercept': True}	124	886	78	882
7	RandomForest	0.544	0.633	{'model__max_depth': 10, 'model__max_features': 'sqrt', 'model__min_samples_split': 5, 'model__n_estimators': 500}	143	985	121	978
8	Dummy_Mean	-0.000	-	-	171	1,006	138	996

Avec des variables binaire les modèles a arbres s'en sortent mieux

Recherche d'hyperparamètre en enlevant les features binaires

Résultats finaux pour y1 (Unité : kBtu) :

	Model	Initial_R2_Test	R2_train	R2_test	Best_R2_CV	Best_Params	MAE_kBtu	RMSE_kBtu	Biais_kBtu	Dispersion_kBtu
0	LinearRegression	0.729	0.742	0.729	0.724	{'model__fit_intercept': True}	3,748,104	23,557,338	1,736,765	23,493,229
1	Ridge	0.703	0.742	0.729	0.724	{'model__alpha': 0.001}	3,748,182	23,558,023	1,736,850	23,493,909
4	Lasso	0.681	0.742	0.728	0.723	{'model__alpha': 0.0001}	3,766,284	23,708,778	1,752,337	23,643,930
3	ElasticNet	0.691	0.739	0.722	0.719	{'model__alpha': 0.001, 'model__l1_ratio': 0.1}	3,825,796	24,041,565	1,794,878	23,974,471
2	SVR	0.703	0.775	0.705	0.741	{'model__C': 50, 'model__gamma': 0.01}	3,843,725	24,531,774	2,141,412	24,438,132
5	XGBoost	0.677	0.797	0.682	0.725	{'model__colsample_bytree': 0.8, 'model__learning_rate': 0.05, 'model__max_depth': 3, 'model__n_estimators': 200, 'model__subsample': 0.8}	4,051,120	24,578,235	2,213,514	24,478,358
6	GradientBoosting	0.676	0.807	0.665	0.722	{'model__learning_rate': 0.05, 'model__max_depth': 3, 'model__n_estimators': 200, 'model__subsample': 0.8}	4,105,721	24,632,282	2,243,102	24,529,937
7	RandomForest	0.646	0.789	0.657	0.681	{'model__max_depth': 10, 'model__max_features': 'sqrt', 'model__min_samples_split': 2, 'model__n_estimators': 500}	4,582,462	26,094,459	3,225,142	25,894,387
8	Dummy_Mean	-0.003	0.000	-0.003	-	-	6,476,160	28,013,201	4,626,670	27,628,489

Résultats finaux pour y2 (Unité : MTCO2e) :

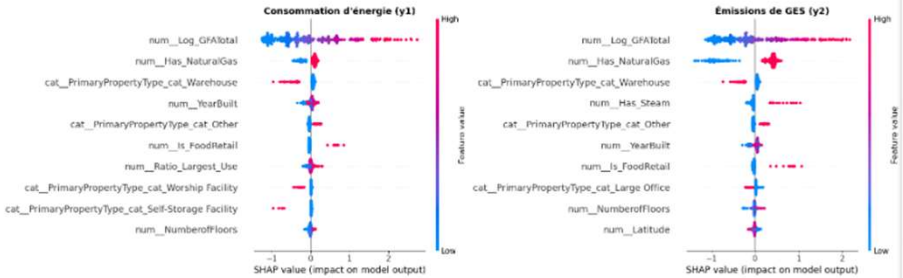
	Model	Initial_R2_Test	R2_train	R2_test	Best_R2_CV	Best_Params	MAE_MTCO2e	RMSE_MTCO2e	Biais_MTCO2e	Dispersion_MTCO2e
0	SVR	0.683	0.780	0.688	0.733	{'model__C': 100, 'model__gamma': 0.01}	130	946	93	941
1	LinearRegression	0.677	0.726	0.677	0.706	{'model__fit_intercept': True}	128	931	84	928
4	Ridge	0.652	0.726	0.677	0.706	{'model__alpha': 0.001}	128	931	84	928
7	Lasso	0.616	0.726	0.676	0.706	{'model__alpha': 0.0001}	129	934	85	930
5	ElasticNet	0.633	0.723	0.672	0.702	{'model__alpha': 0.001, 'model__l1_ratio': 0.1}	130	942	87	938
2	XGBoost	0.662	0.819	0.670	0.715	{'model__colsample_bytree': 0.8, 'model__learning_rate': 0.05, 'model__max_depth': 5, 'model__n_estimators': 100, 'model__subsample': 0.8}	134	954	101	949
3	GradientBoosting	0.661	0.796	0.668	0.716	{'model__learning_rate': 0.05, 'model__max_depth': 3, 'model__n_estimators': 200, 'model__subsample': 0.8}	132	953	98	948
6	RandomForest	0.624	0.773	0.626	0.682	{'model__max_depth': 10, 'model__max_features': 'sqrt', 'model__min_samples_split': 5, 'model__n_estimators': 500}	141	978	117	971
8	Dummy_Mean	-0.000	0.000	-0.000	-	-	171	1,006	138	996

En retirant plusieurs binaires les modèles a régression

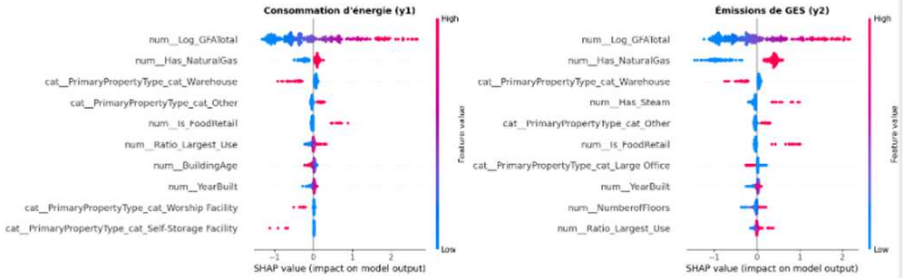
15.Vérification de l'impact positif et négatif des features sur les différents groupes de modèles (Régression, Arbres, A noyau)

Modèles à base d'arbres (Tree-based)

Génération : XGBoost

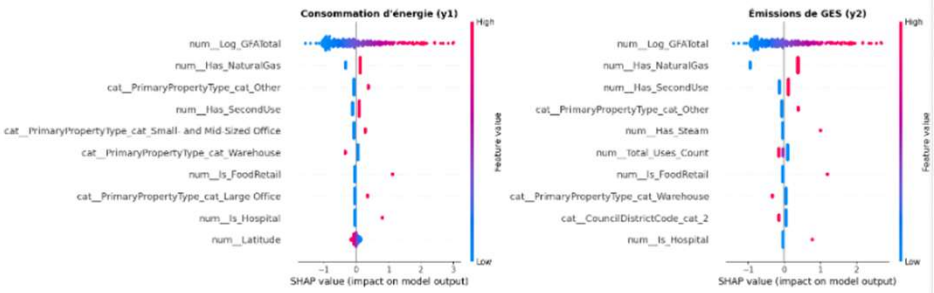


Génération : GradientBoosting

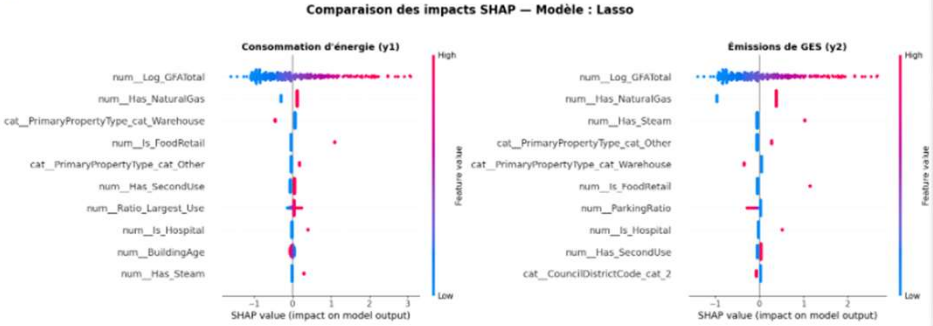


Modèles linéaires (Linear)

Génération : Ridge



Génération : Lasso



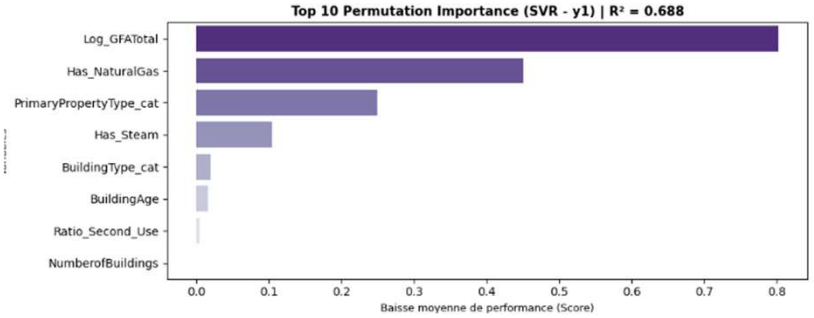
Génération : ElasticNet

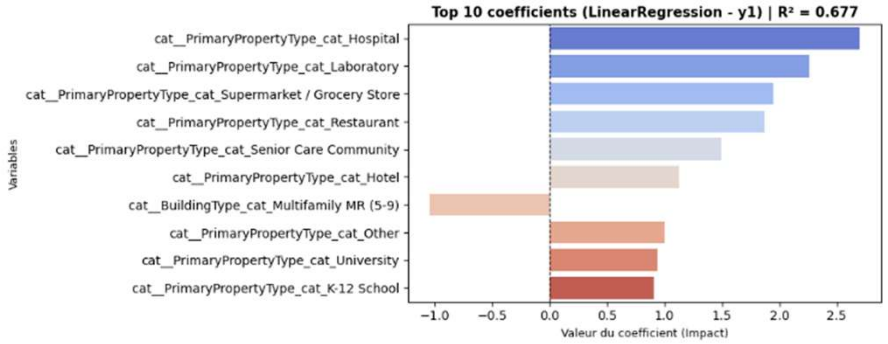
16.Conclusion

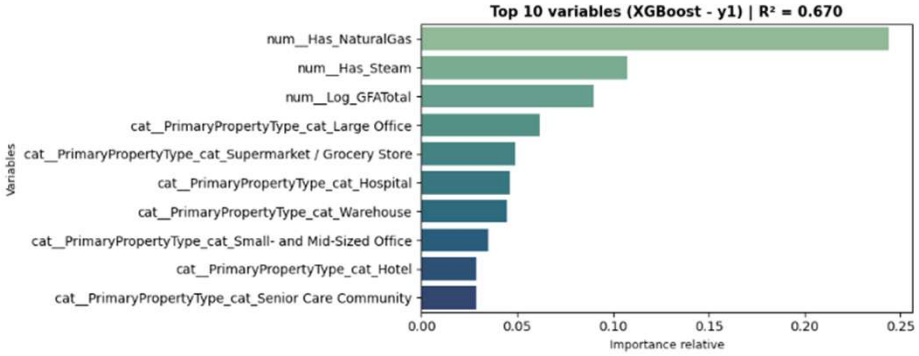
L'analyse comparative des graphiques SHAP et des tableaux de performance (`y1` pour l'énergie et `y2` pour les GES) met en lumière des contrastes marqués entre les familles de modèles.

1. Stabilité et consensus sur les variables clés (Le tronc commun)

Selon la famille de modèle utilisée, l'importance accordée aux variables explicatives diffère sensiblement pour cette étude.







2. Cibles
Deux cibles : y1 (consommation d'énergie) et y2 (émissions de GES), toutes deux transformées en log1p pour stabiliser leur distribution très asymétrique.

3. Modélisation
8 modèles comparés : Dummy, LinearRegression, Ridge, Lasso, ElasticNet, SVR, RandomForest, GradientBoosting, XGBoost.
Évaluation via cross-validation + test set, avec métriques R^2, MAE et RMSE (en échelle log et en échelle réelle).

4. Split et prétraitement
Split train/test fait avant toute transformation, pour éviter le data leakage.
ColumnTransformer : StandardScaler pour le numérique, OneHotEncoder pour le catégoriel.

5. Bug majeur identifié et corrigé
Les objets modèles étaient partagés (même instance) entre l'entraînement sur y1 et y2 → le second écrasait le premier en mémoire. Corrigé avec clone() sur le modèle et le préprocesseur à chaque appel

6. Diagnostic des erreurs
Visualisation prédit vs réel, distribution des erreurs par catégorie (seuil médian), et analyse des résidus extrêmes.
Identification d'un pattern : hôpitaux et campus universitaires systématiquement sous-estimés (grosses erreurs concentrées sur ces catégories).

7. Interprétabilité
Analyse SHAP et permutation importance sur les 8 modèles, comparant modèles linéaires (Ridge en tête, $R^2 \approx 0.75$) vs modèles à arbres.
Mise en évidence que Log_GFATotal et les indicateurs d'énergie (Has_NaturalGas, Has_Steam) sont les variables dominantes, mais avec des hiérarchies différentes selon la famille de modèle.

Résultat final :

SVR (Kernel)obtient le meilleur score global ($R^2 \approx 0.74$), ce qui signifie qu'il explique le mieux, en moyenne, les variations de consommation d'énergie entre les bâtiments. Mais ce score global cache une réalité : ses prédictions concrètes, en kBtu réels, sont légèrement moins précises que celles d'un modèle plus simple.

La régression linéaire (LinearRegression / Ridge), à l'inverse, obtient un score global très proche ($R^2 \approx 0.72$) mais se distingue par des **erreurs de prédiction plus faibles en valeur réelle,** c'est-à-dire que quand on regarde concrètement "de combien le modèle se trompe" pour un bâtiment donné, elle est plus précise que SVR. Elle a aussi l'avantage d'être plus simple, plus rapide et plus facile à expliquer : on peut directement voir quelle variable pousse la prédiction à la hausse ou à la baisse.

En résumé : il n'y a pas un unique "meilleur modèle" universel, le bon choix dépend de la question posée. Si l'objectif est de capturer le maximum de variance globale, SVR est légèrement en tête. Si l'objectif est de minimiser l'erreur réelle sur chaque prédiction individuelle la régression linéaire est un choix au moins aussi bon, tout en étant plus simple à déployer et à interpréter et si l'on veut complexifier, les modèles a arbres (XGBoost, GradientBoosting) prennent l'avantage.